uc3m | Universidad Carlos III de Madrid

ROBOTICSLAB

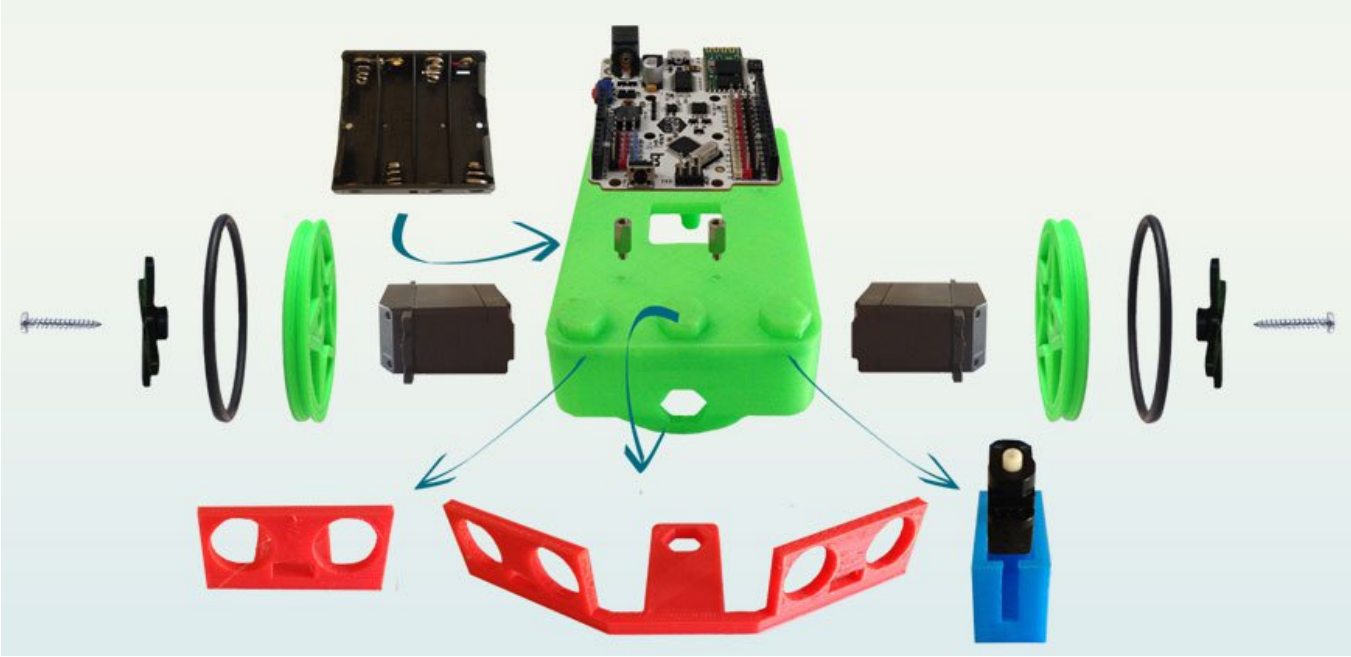
Robotics Laboratory

Foundations of Technology
International Foundation Program

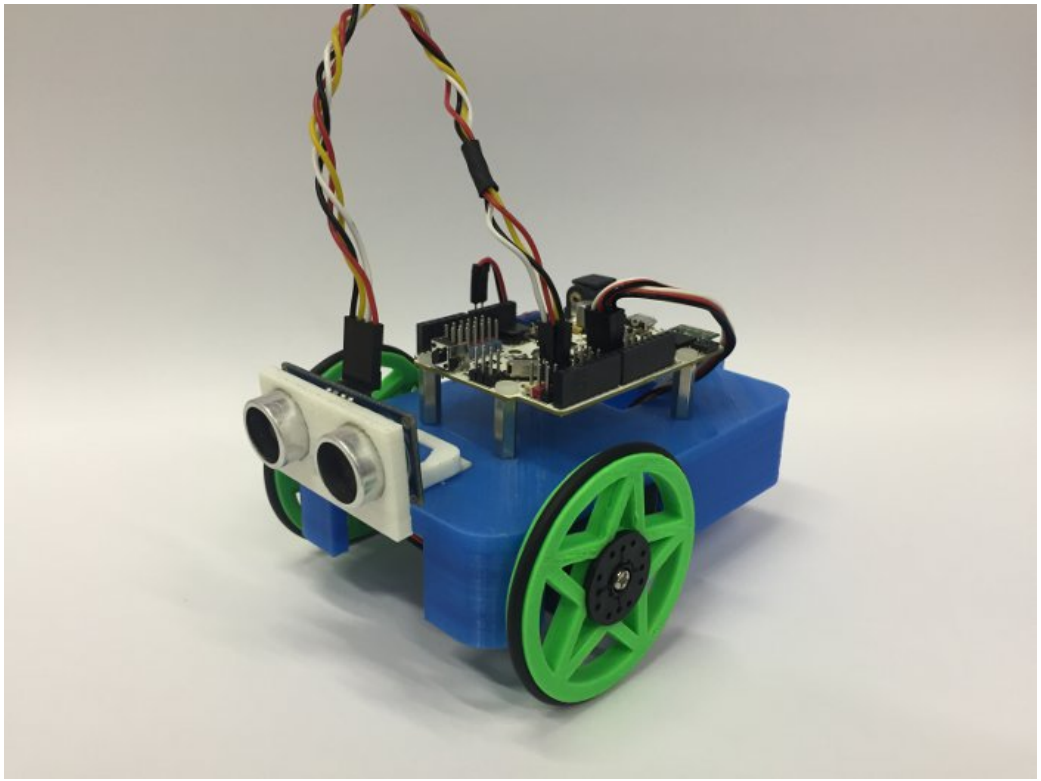
Lab schedule

Wednesday	Thursday
	29/01/26 Lab
4/02/23 Lab	5/02/26 Lab
11/02/23 Lab	12/02/26 Lab
18/02/23 Lab	19/02/26 Lab
25/02/23 Lab	26/02/26 Demo

Our robot?



Our robot



Basic concepts I

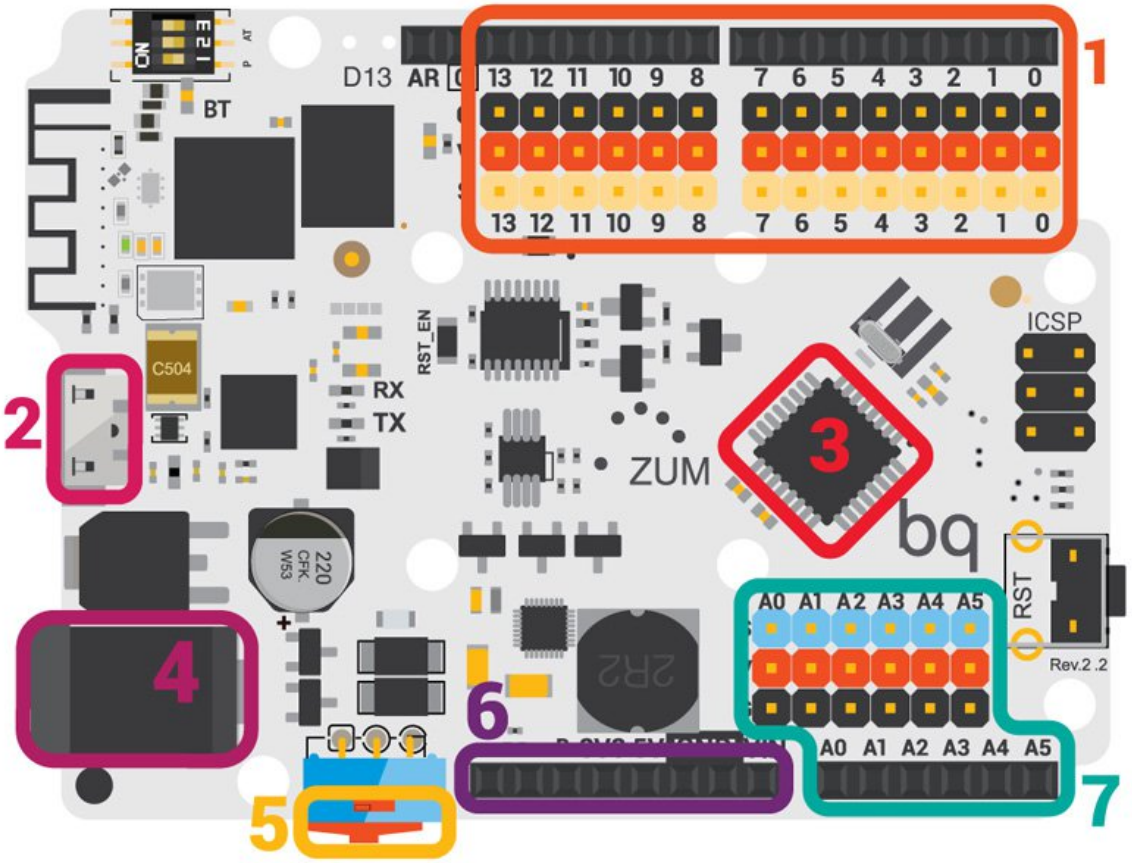
- Sensors
  - ★ Perception of the environment. What is the intensity of the light? Is there any object close enough? What is the temperature in this room?
- Actuators
  - ★ Perform an action that changes the environment. For example, turn a light on, make a noise, or move around.
  - ★ Actuators are operated by commands that define how to conduct the action.
- Control board
  - ★ Electronic device that receives and processes data and instructions which have been programmed in its memory.
  - ★ It relates the data obtained from the sensors with the actions executed by the actuators.

Basic concepts II

- Digital
  - ★ A digital device distinguishes just 2 values (1 or 0, true or false).
  - ★ For instance, a button can be pressed or release.
- Analogic
  - ★ An analog device *understands* a range of values (more than 2).
  - ★ For example, a noise sensor informs about the intensity of the ambient sound with a value.

Control board

- Zum Core BT-328

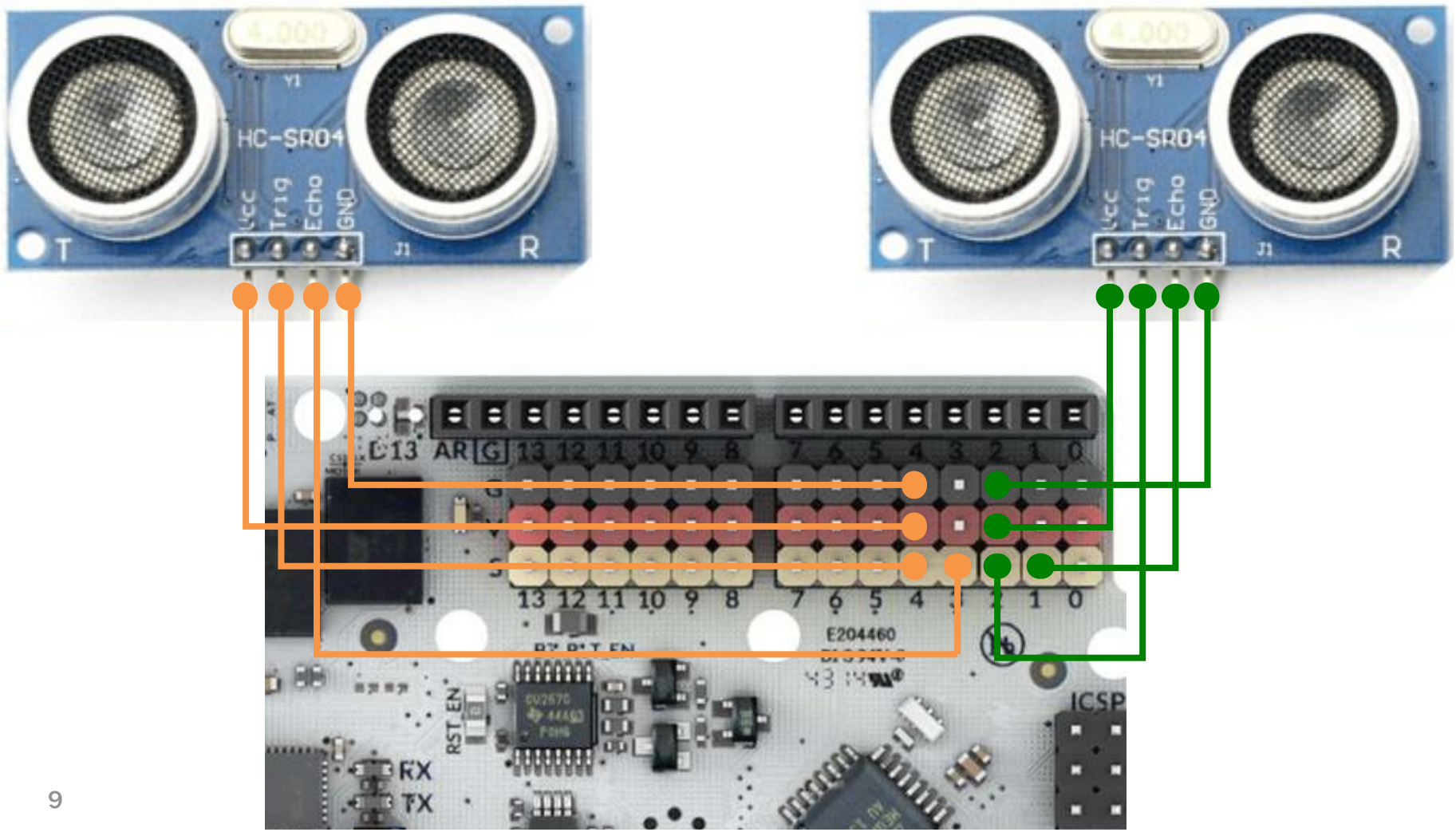


1. Digital pins
2. USB port
3. Processor
4. Power
5. on/off
6. Power pins
7. Analog pins

Sensors

- Ultrasonic sensor HC-SR04
- It measures distances based on the time gap between the emitter emits an audio signal and the receptor receives the echo.
- Problems?
  - ★ Multiple echoes
  - ★ Signal mitigation
  - ★ Low accuracy

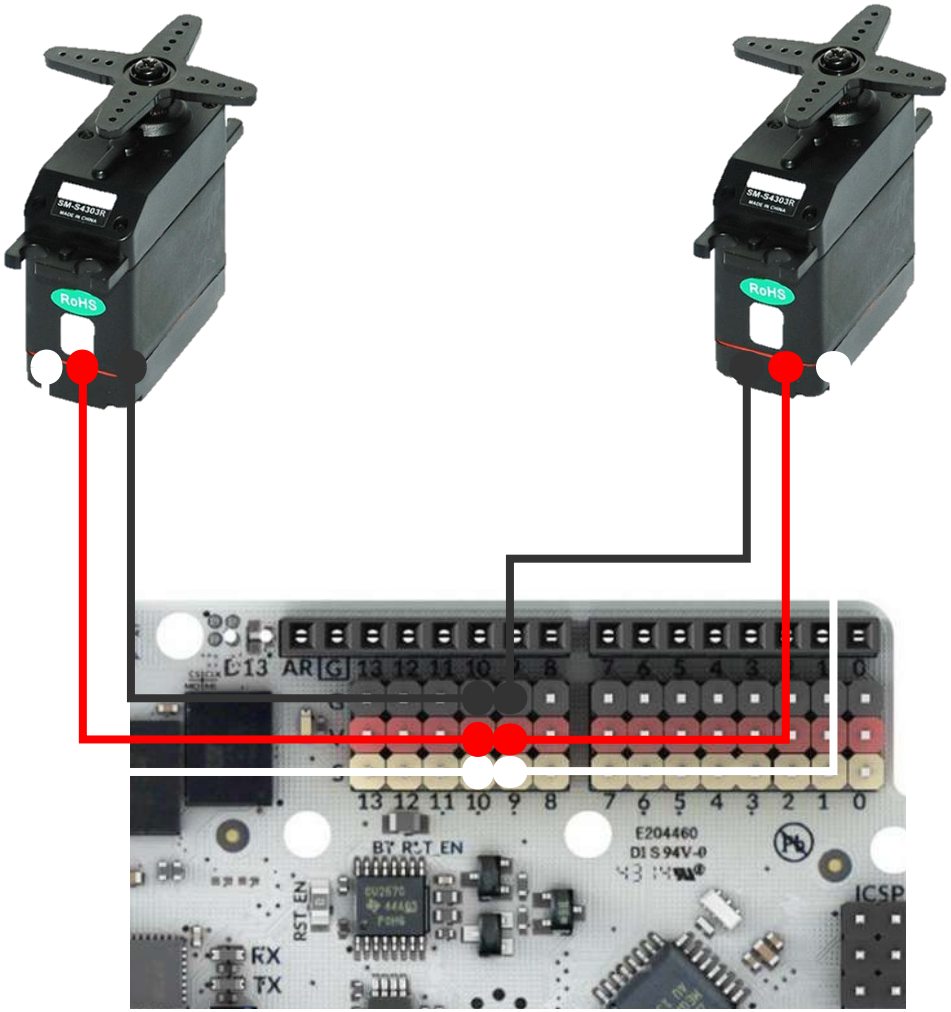
Connection


HC-SR04
Vcc
Trig
Echo
GND
T
R
HC-SR04
Vcc
Trig
Echo
GND
T
R
13 12 11 10 9 8
7 6 5 4 3 2 1 0
E204460
ICSP
RST EN
RX
TX

Actuators

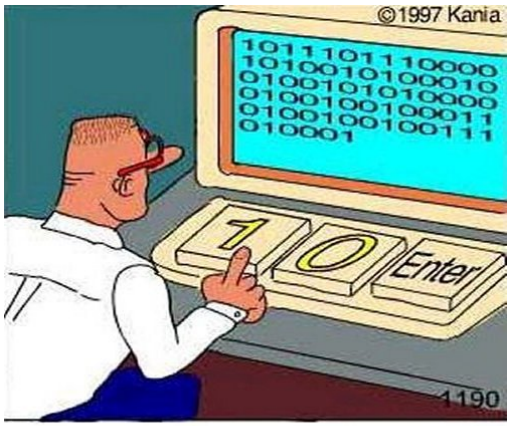
- Servomotor x 2
- Direction of rotation and velocity
- How do you make your robot to turn right and left?

Connection



Adding intelligence to our robots

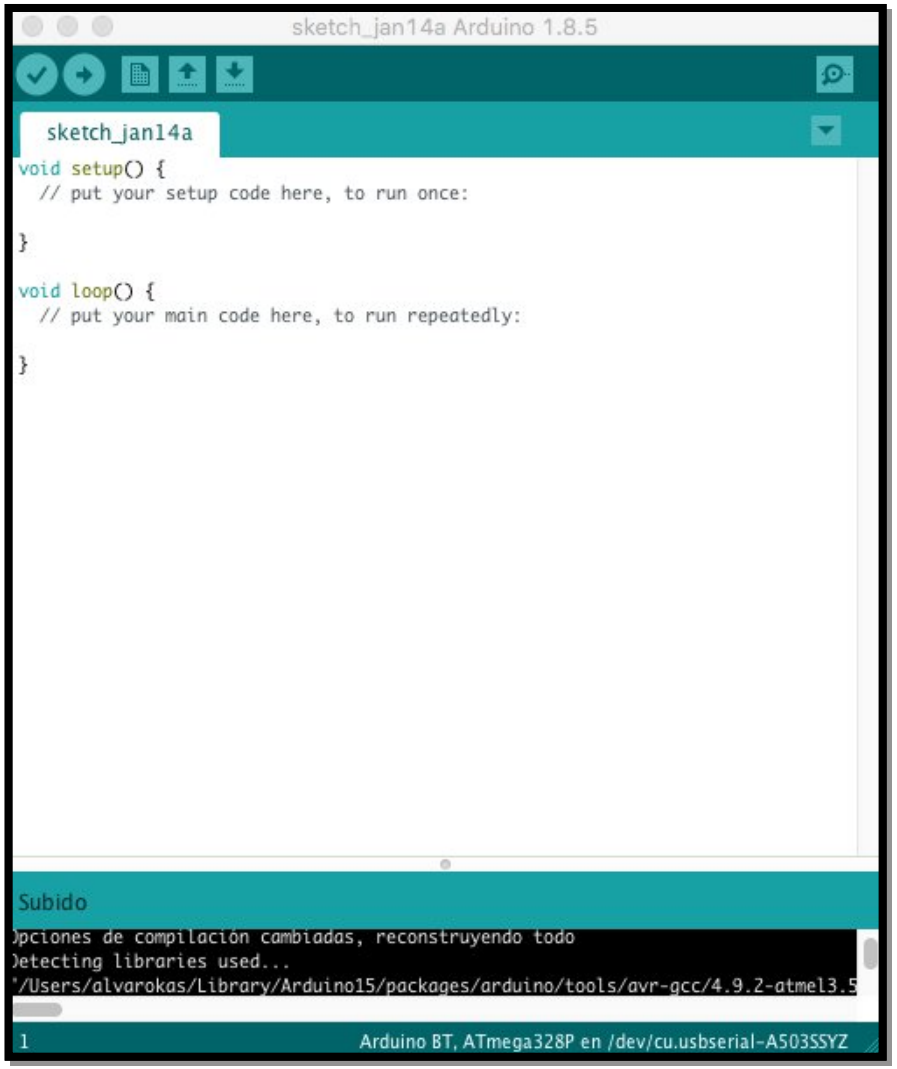
- Programming



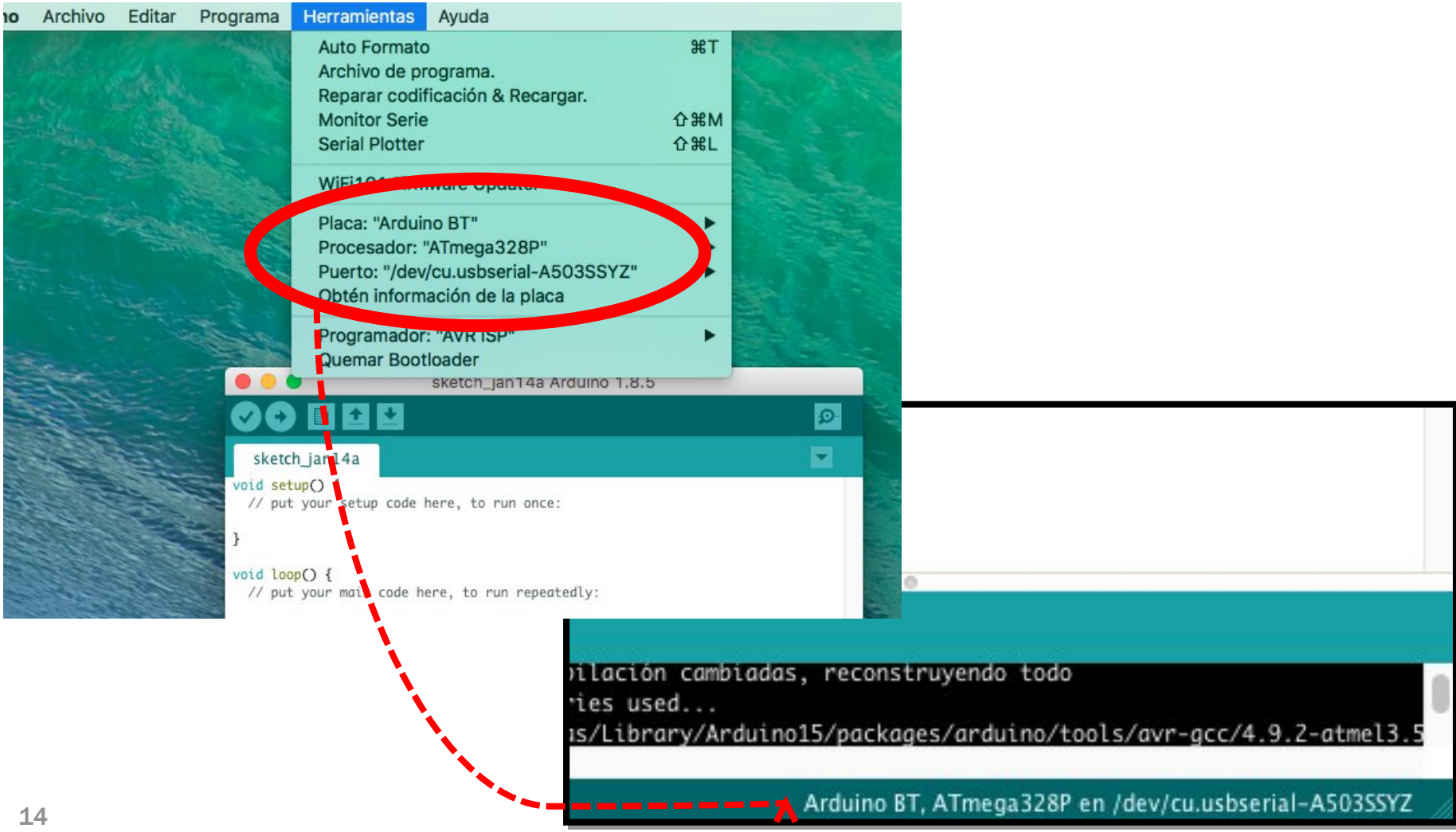
LOS VERDADEROS PROGRAMADORES PROGRAMAN EN BINARIO

- Integrated development environment
  - ★ Arduino IDE
    https://www.arduino.cc/en/main/software

Arduino IDE

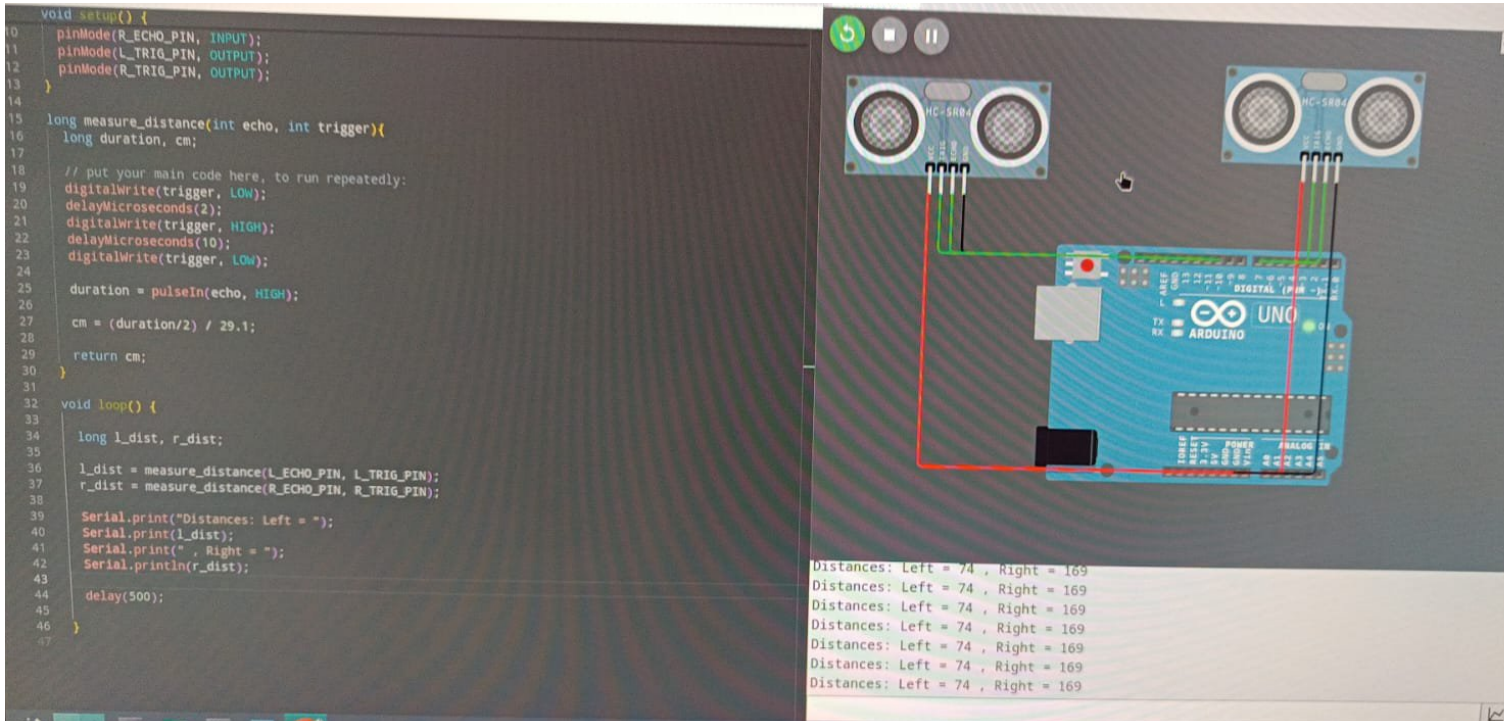

sketch_jan14a Arduino 1.8.5
sketch_jan14a
void setup() {
// put your setup code here, to run once:
}
void loop() {
// put your main code here, to run repeatedly:
}
Subido
Opciones de compilación cambiadas, reconstruyendo todo
Detecting libraries used...
"/Users/alvarokas/Library/Arduino15/packages/arduino/tools/avr-gcc/4.9.2-atmel3.5
1
Arduino BT, ATmega328P en /dev/cu.usbserial-A503SSYZ

Arduino IDE: configuration

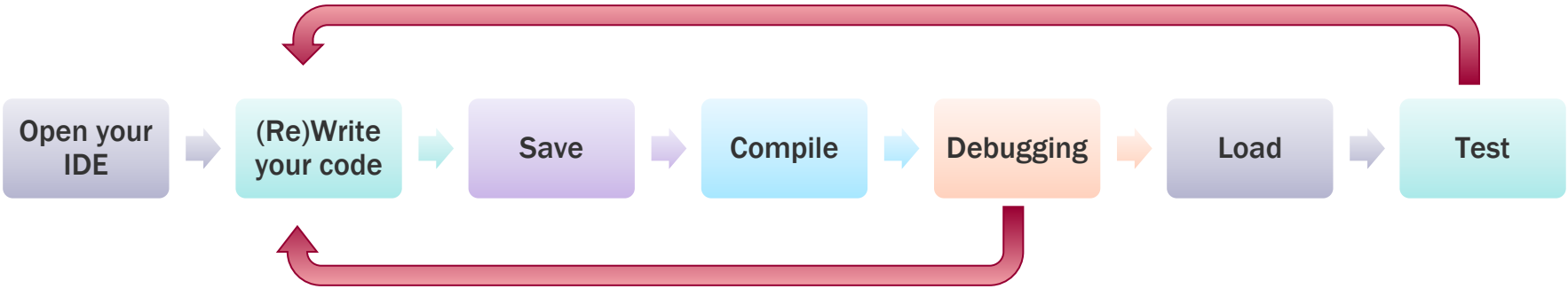


Wokwi: Arduino simulator

- https://wokwi.com/



Workflow



Don't forget to think!!

Learning how to program Arduino

- Your best friends
  - ★ www.arduino.cc
  - ★ Google
  - ★ Serial.print + Serie monitor

Arduino: skeleton

- *setup* and *loop*
- Tic-tac example

```
void setup() {
  // put your setup code here, to run once:

  Serial.begin(9600);
  Serial.println("Hola!!!");
}

void loop() {
  // put your main code here, to run repeatedly:

  Serial.println("tic"); //print on the serial port
  delay (1000); //wait 1000ms

  Serial.println("tac"); //print on the serial port
  delay(1000); //wait 1000ms
}
```

Arduino: variables

- Piece of memory where we store a value and refer to it by a given name.
- Types: char, byte, int, float, String
  https://www.arduino.cc/reference/en/#variables
- How to use it:

  [type] [name];
  [type] [name] = [initial value];
  [name] = [new value];

Arduino: variables

```
char un_caracter = 'a';   //declaración de la variable un_caracter donde se almacena la letra 'a'
byte un_numero = 123;     //declaración de la variable un_numero donde se almacena el entero 123
float un_decimal = 12.345;//declaración de la variable un_decimal donde se almacena el número decimal 12.345

void setup() {
  // put your setup code here, to run once:

  un_numero = 1234; //asignando un valor fuera del rango válido. Ver mensaje del compilado.
  unsigned int a = 12;
  unsigned int b = 34;
  Serial.begin(9600);
  Serial.print("a + b = ");
  Serial.print(a);
  Serial.print(" + ");
  Serial.print(b);
  Serial.print(" = ");
  Serial.println(a+b);
}
```

Arduino: variables

- Exercise: change the Tic-Tac program to include 2 String variables used to print “tic” and “tac”. Besides, use a variable where the waiting time is defined.

Arduino: variables

- Answer

```
String s1 = "TIC";
String s2 = "TAC";

void loop() {
  // put your main code here, to run repeatedly:

  Serial.println(s1); //print on the serial port
  delay (1000); //wait 1000ms

  Serial.println(s2); //print on the serial port
  delay(1000); //wait 1000ms
}
```

Arduino: functions

- Fragment of code that is referred by a given name.
- Benefits:
  - ★ Better structured and tidier code.
  - ★ Reuse of code
- How to use it
  - ★ Declaration

```
[type of the result] [name of the function] ( [types of the parameters] )
{
        // code
}
```

  - ★ Calling

```
[name of the function] ( [value of the parameters] ) ;
```

Arduino: functions

```
/*
 * Función que imprime un saludo por la comunicación serie
 */
void saludar() {
  Serial.println("Hola, buenos días");
}

/*
 * Función que devuelve el resultado de multiplicar 2 números en coma flotante
 */
float multiplicar(float f1, float f2)
{
  float resultado = f1*f2;
  return resultado;
}

void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600); //apertura de la comunicación seria a 9600 baudios

  float f = multiplicar(12.34, 56.78);
  Serial.print("El resultado es ");
  Serial.println(f);
}

int espera = 2000;

void loop() {
  // put your main code here, to run repeatedly:
  saludar();
  delay(espera);
}
```

Arduino: functions

- Exercise: Most Arduino boards have a pin connected to an on-board LED in series with a resistor. The constant LED_BUILTIN is the number of the pin to which the on-board LED is connected. Write a function *read_led_builtin* that prints the value of the constant LED_BUILTIN and the value of that pin.

Arduino: FUNCIONES

- Solución

```
// Function to print the value of the constant
// LED_BUILTIN to the serial port
// Returns: void
// Parameters: void
void print_LED_BUILTIN() {
  Serial.print ("LEB_BUILTIN pin = ");
  Serial.println (LED_BUILTIN);
  Serial.print ("LEB_BUILTIN value = ");
  int val = digitalRead(LED_BUILTIN);
  Serial.println (val);
}

void setup() {
  // put your setup code here, to run once:

  Serial.begin(9600);
  print_LED_BUILTIN();
}

void loop() {
  // put your main code here, to run repeatedly:

}
```

Arduino: control of the execution

- Conditional instructions
  - ★if … else
  - ★switch … case

```
if (x > 100) {
        Serial.println("x es mayor de 100");
}
else {
        Serial.println("x no es mayor de 100")
}
```

```
switch (var) {
case val1:
        // código
        break;
case val2:
        // código
        break;
default:
        // código
        break;
}
```

Arduino: control of the execution

- Loops
  - ★for

```
for (int i = 0; i < 10; i++)
{
        Serial.print("Intento ");
        Serial.println (i);
}
```

  - ★do…while

```
do {
        x = read_sensors(); //this function doesn't exis
} while (x<100);
```

  - ★while

```
var=0
while (var < 200) {
        // hacer algo 200 vece
        var++;
}
```

Arduino: exercise I

- Write a function that receives an integer parameter and print it to the Serial port if it is an even number. Otherwise, it prints an informative message. The function returns a boolean value: true if the parameter is even or false if the parameter is an odd number.

Arduino: exercise I

- Answer

```
// Function that sends the number received as a
// parameter over the serial port if it is an even number.
bool send_if_even(int n)
{
  if (n%2 == 0) // n is even
  {
    Serial.println(n);
    return true;
  } else { // n is odd
    Serial.println("The number is not even");
    return false;
  }
}

void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);

  for (int i = 0; i<5; i++) {
    if (send_if_even(i))
      Serial.println("Even number sent");
  }
}

void loop() {
  // put your main code here, to run repeatedly:

}
```

Arduino: exercise II

- Write a function that stops the execution of the program until any data is received through the serial port.
- Tip: Serial.available()
  https://www.arduino.cc/reference/en/language/functions/communication/serial/available/

Arduino: exercise II

```
// This function waits till new data has arrived in the Serial port
void wait_for_data()
{
  while (Serial.available()==0) {
    Serial.print(".");
    delay(1000); //waiting 1s before we check it again
  }
}

void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);
  Serial.println("Serial communication ready");
}

void loop() {
  // put your main code here, to run repeatedly:
  Serial.println("Waiting for data...");
  wait_for_data();
  Serial.println("Data received!");

}
```

Arduino: exercise III

- Change the previous code in order to read and print the characters from the serial port.
- Tip: Serial.read()

Arduino: Ejercicio III

- Answer

```
// This function waits till new data have arrived in the Serial port
void wait_for_serial_data()
{
  while (Serial.available()==0) {
    Serial.print(".");
    delay(1000); //waiting 1s before we check it again
  }
}

// This function reads data from the Serial port
// Returns the number of bytes read
int read_serial_data() {
  int counter = 0; // this variable is used to count the number of bytes read

  while (Serial.available()!=0)
  {
    byte b = Serial.read();
    Serial.println(b);
    counter++;
  }
  return counter; //the number of bytes is returned
}

void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);
  Serial.println("Serial communication ready");
}

void loop() {
  // put your main code here, to run repeatedly:
  Serial.print("Waiting for data ");
  wait_for_serial_data();
  Serial.println("Data received!");

  Serial.println("Reading serial data");
  int num_bytes = read_serial_data();
  Serial.print(num_bytes);
  Serial.println(" read!");
}
```

Actuators: LEDs

- Digital output used to switch a LED on or off.
- How to use it:
  1. Seting up the pin

     ```
     pinMode(13, OUTPUT);
     ```

  2. Control

     ```
     digitalWrite(13, HIGH);
     digitalWrite(13, LOW);
     ```

Watch out! In order to write (i.e. change the state) of a pin, first you need to configure it as an output.

Actuators: LEDs

- Exercise1: change the Tic-Tac program so the LED connected to pin 13 turns on with the “tic” and it turns off with the “tac”.

Actuators: LEDs

- Exercise1: answer

```
void setup() {
  // put your setup code here, to run once:

  Serial.begin(9600);
  pinMode (13, OUTPUT); // seting up pin 13 as an output

}

String s1 = "TIC";
String s2 = "TAC";
int t = 2000;

void loop() {
  // put your main code here, to run repeatedly:

  digitalWrite(13, HIGH);
  Serial.println(s1); //print on the serial port
  delay (t); //wait 1000ms

  digitalWrite(13, LOW);
  Serial.println(s2); //print on the serial port
  delay(t); //wait 1000ms
}
```

Actuators: LEDs

- Exercise 2: create a program to control the LED connected to pin 13 using the serial port: the character ‘H’ turns it on, and the character ‘L’ turns it off.

Actuators: LEDs

- Exercise 2: answer

```
// This function reads data from the Serial port
// Returns the number of bytes read
void process_serial_data() {
  while (Serial.available()!=0)
  {
    char b = Serial.read();
    if (b=='H' || b=='h')
    {
      digitalWrite(LED_BUILTIN, HIGH);
      Serial.println("LED ON");
    }
    else if(b=='L' || b=='l')
    {
      digitalWrite(LED_BUILTIN, LOW);
      Serial.println("LED OFF");
    }
  }
}

void setup() {
  // put your setup code here, to run once:
  pinMode(LED_BUILTIN, OUTPUT);

  Serial.begin(9600);
  Serial.println("Serial communication ready");
}

void loop() {
  // put your main code here, to run repeatedly:
  process_serial_data();
}
```

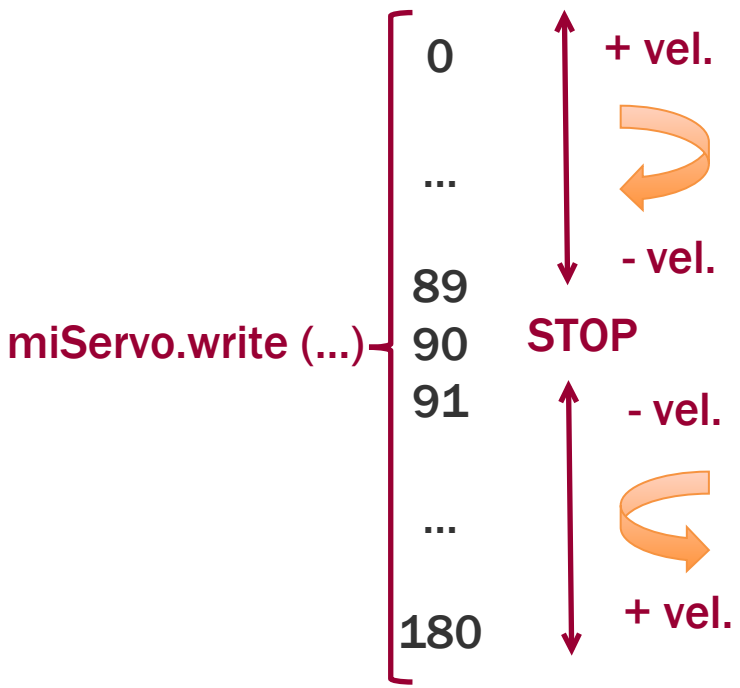
Actuators: servomotors

- ★ Regular ones: position control
- ★ **<u>Continuous rotation</u>**: velocity control

- How to use it: library Servo
  1. Configure the control pin for the servo
     miServo.attach ([pin number]);
  2. Command the motor
     miServo.write ([value]);

Actuators: continuous rotation servo

- Library Servo
  https://www.arduino.cc/en/Reference/Servo




```
#include <Servo.h>

Servo miServo; //variable para controlar el servo

void setup()
{
        miServo.attach(10);
}
void loop()
{
        miServo.write(90); //manda el servo la orden de parar
```

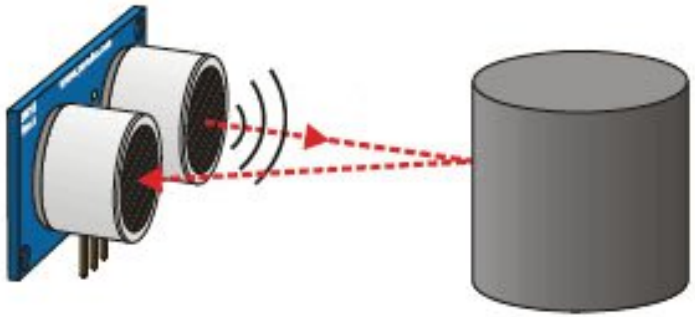
Actuators: continuous rotation servo

- Exercise: create a program to control the motion of the robot using the serial port. Consider 5 commands: move forward, move backward, turn right, turn left, and stop.

Don't forget to turn the robot ON!!!

Sensors: sonars

- HC-SR04 ultrasonic sensor
- Provides the time gap from the emitting and reception of a sonic pulse (microseconds).
  - ★ Speed of sound* = 343m/s = 1cm/29.2us
  - ★ Distance = speed of sound * tiempo / 2



Sensors: sonars

- HC-SR04 ultrasonic sensor
- Range of operation: 2cm … 400cm
- Pins
  - ★VCC (5V)
  - ★GND
  - ★Trigger (output)
  - ★Echo (input)

Sensors: sonars

```
#define TRIG_PIN 4
#define ECHO_PIN 3

void setup() {
  // put your setup code here, to run once:

  // serial port begin
  Serial.begin(9600);

  // setting up inputs and outputs
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);

}
```

```
void loop() {
  // put your main code here, to run repeatedly:
  long duration, cm;

  digitalWrite(TRIG_PIN, LOW);  // first, trig pin is set to low
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH); // Then, it is set to high...
  delayMicroseconds(10);        // ...during 10us
  digitalWrite(TRIG_PIN, LOW);  // Finally, it is set to low again

  // Read the signal from the sensor: a HIGH pulse whose
  // duration is the time (in microseconds) from the sending
  // of the ping to the reception of its echo off of an object.
  duration = pulseIn(ECHO_PIN, HIGH);

  // convert the time into distance
  // distance = time * velocity / 2
  cm = (duration/2) / 29.1;

  Serial.print(cm);
  Serial.println("cm");

  delay(500);

}
```

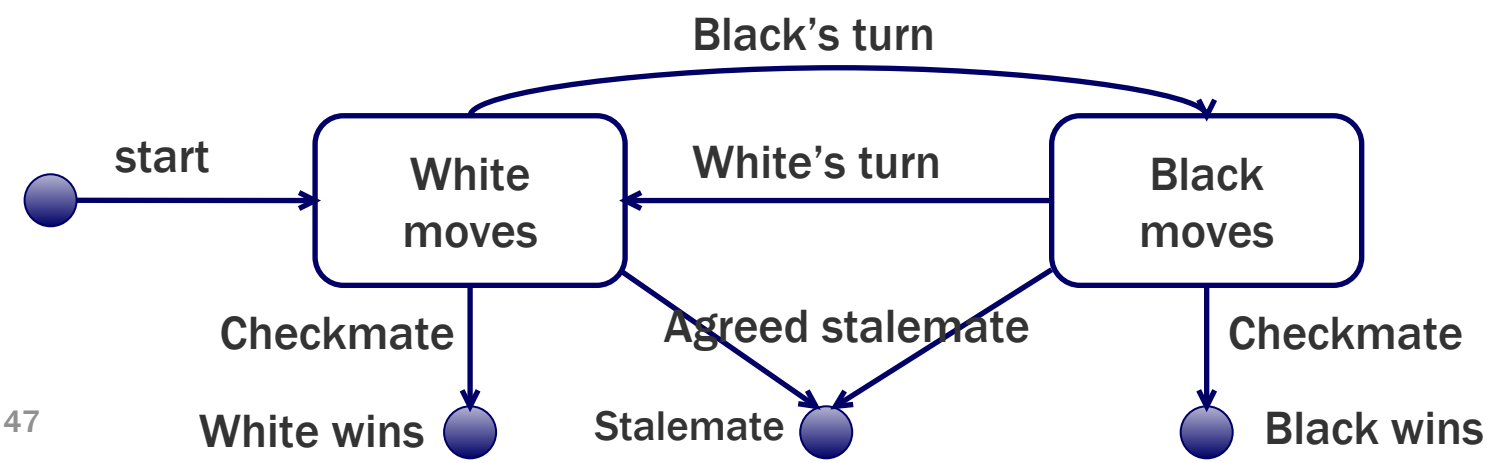
Sensors: sonar

- Exercise: create a program that, using the information provided by the sonars, makes the robot to avoid obstacles.

Don't forget to turn the robot ON!!!

State machine

- Model of operation of a system where the outputs depend on the current inputs and also on the past inputs (which determine the state).

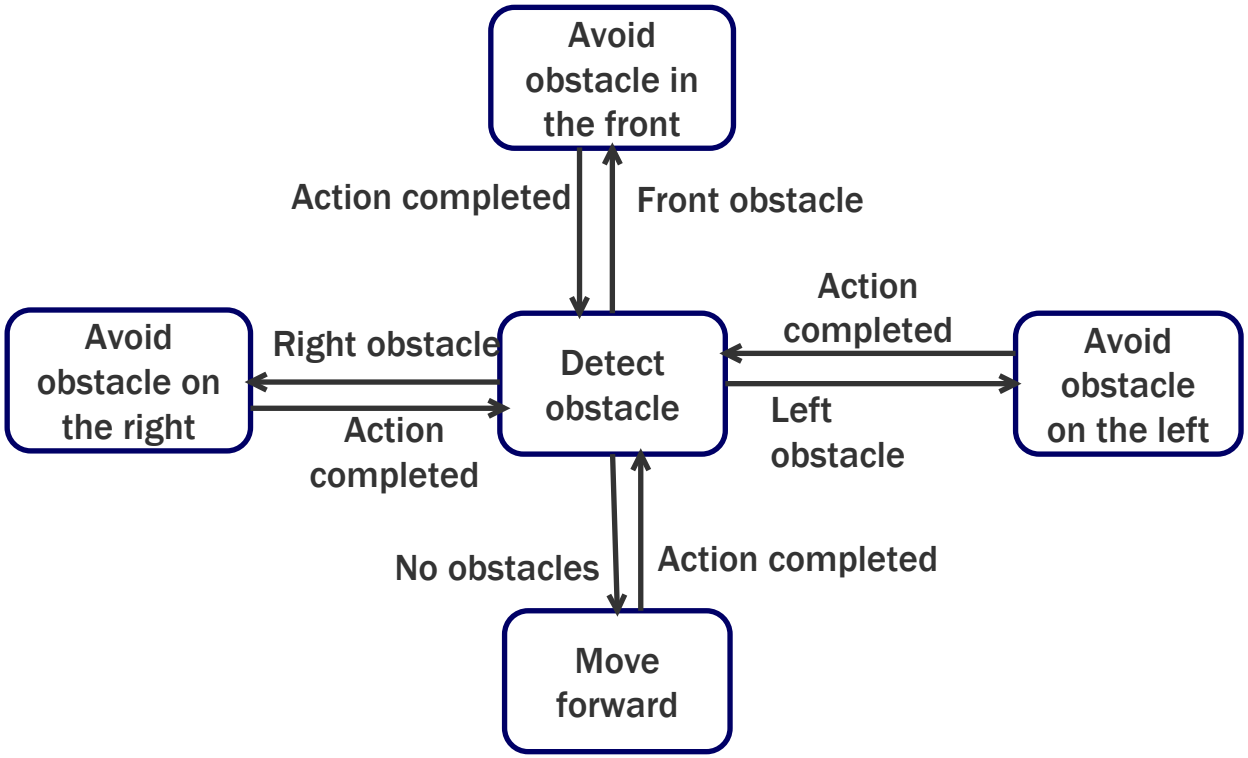


State machine

- Exercise: define a state machine that makes your robot to move around safely. Code it and test it in the robot.

State machine

- Answer: this is a possible solution



State machine

- Answer skeleton (part I)

```
// State machine to avoid obstacles
// Possible states are: move forward, avoid obstable in front,
// avoid obstacle on the right, avoid obstacle on the left, and detect obstacle.

int obstacle = 0; //variable used to store the position of the obstacle
// 0 = no obstacle, 1=obstacle in front, 2=obstacle on the right, and 3=obstacle on the left

int SAFETY_DISTANCE = 10; // safety distance is set to 10 cms.
// Any object closer than 10 cm is considered as an obstacle.

// Function used to detect obstacles.
// Returns: 0 if obstacles are not detected, 1 if an obstacle is
// detected in front of the robot, 2 if an obstacle is detected
// on the right, or 3 if an obstacle is detected on the left side.
// Parameters: distance: minimum distance to consider an obstacle
int detect_obstacle(int distance) {
  int v;

  // ... your code goes here ...

  return v;
}

void setup() {
  // put your setup code here, to run once:

  // detection of obstacles at the very beginning
  obstacle = detect_obstacle (SAFETY_DISTANCE);

}
```

State machine

- Answer skeleton (part II)

```
void loop() {
  // put your main code here, to run repeatedly:

  switch (obstacle)
  {
    case 0: // no obstacle -> state: move forward
            // call move_forward function
            break;
    case 1: // obstacle in front -> state: avoid front obstacle
            // call avoid_front_obstacle function
            break;
    case 2: // obstacle on the right -> state: avoid right obstacle
            // call avoid_right_obstacle function
            break;
    case 3: // obstacle on the left -> state: avoid left obstacle
            // call avoid_left_obstacle function
            break;
    default:// unexpected value of state
            // print a warning message
            break;
  }

  // When the other states are completed -> state: detect obstacle
  obstacle = detect_obstacle (SAFETY_DISTANCE);

}
```

Now…are you able to make your robot scape from a maze?

Enrique Fernández Rodicio
enrifern@ing.uc3m.es
Universidad Carlos III de Madrid